EVM Circuit

개요

EVM Circuit은 ZKEVM 구현체의 핵심 회로로, 이더리움 가상 머신(EVM)의 실행 과정을 검증합니다. 이 회로는 각 opcode의 실행이 올바르게 수행되었는지 확인하고, 상태 전환이 정확한지 검증합니다.

주요 역할:

- EVM opcode 실행의 정확성 검증
- 스택, 메모리, 스토리지 연산 검증
- 가스 소비 계산 검증
- 실행 상태 간 전환 검증
- 다른 회로(State, Bytecode, Copy 등)와의 룩업을 통한 데이터 일관성 보장

파일 위치:

- 메인 파일: `zkevm-circuits/src/evm_circuit.rs`
- 실행 로직: `zkevm-circuits/src/evm_circuit/execution.rs`
- 단계 정의: `zkevm-circuits/src/evm_circuit/step.rs`

ExecutionState 열거형

`ExecutionState` 는 EVM 실행의 각 단계를 나타내는 열거형입니다. 각 상태는 특정 opcode 또는 내부 상태를 나타냅니다.

정의 위치: `zkevm-circuits/src/evm_circuit/step.rs`

내부 상태

```
pub enum ExecutionState {
// 내부 상태
BeginTx, // 트랜잭션 시작
EndTx, // 트랜잭션 종료
EndInnerBlock, // 내부 블록 종료
EndBlock, // 블록 종료
Padding, // 패딩 (빈 행 채우기)
// ... opcode 상태들
}
```

Opcode 성공 케이스

각 opcode 또는 opcode 그룹에 대응하는 실행 상태:

- **산술 연산**: `ADD_SUB`, `MUL_DIV_MOD`, `SDIV_SMOD`, `ADDMOD`, `MULMOD`, `EXP`
- **비트 연산**: `SHL_SHR`, `SAR`, `BITWISE` (AND, OR, XOR), `NOT`, `BYTE`
- **비교 연산**: `CMP` (LT, GT, EQ), `SCMP` (SLT, SGT), `ISZERO`
- **환경 정보**: `ADDRESS`, `BALANCE`, `ORIGIN`, `CALLER`, `CALLVALUE`, `GASPRICE`, `CHAINID`
- **블록 정보**: `BLOCKHASH`, `BLOCKCTXU64`, `BLOCKCTXU160`, `BLOCKCTXU256`
- **스택 연산**: `POP`, `PUSH`, `DUP`, `SWAP`
- **메모리 연산**: `MEMORY` (MLOAD, MSTORE, MSTORE8), `MCOPY`, `MSIZE`
- **스토리지 연산**: `SLOAD`, `SSTORE`, `TLOAD`, `TSTORE`
- **제어 흐름**: `JUMP`, `JUMPI`, `JUMPDEST`, `PC`, `GAS`, `STOP`

- **호출 연산**: `CALL_OP` (CALL, CALLCODE, DELEGATECALL, STATICCALL), `CREATE`, `CREATE2`
- **반환 연산**: `RETURN_REVERT`, `SELFDESTRUCT`
- **로그**: `LOG` (LOG0-LOG4)
- **기타**: `SHA3`, `CALLDATACOPY`, `CODECOPY`, `EXTCODECOPY`, `RETURNDATACOPY`

에러 상태

실행 중 발생할 수 있는 에러 상태:

- `ErrorInvalidOpcode` : 유효하지 않은 opcode
- `ErrorStack` : 스택 언더플로우/오버플로우
- `ErrorWriteProtection` : 쓰기 보호 위반
- `ErrorInvalidCreationCode` : 유효하지 않은 생성 코드
- `ErrorInvalidJump` : 유효하지 않은 점프
- `ErrorReturnDataOutOfBound` : 반환 데이터 범위 초과
- `ErrorOutOfGas*` : 다양한 가스 부족 에러

프리컴파일

프리컴파일 호출 상태:

- `PrecompileEcrecover` : ECDSA 서명 복구
- `PrecompileSha256` : SHA256 해시
- `PrecompileRipemd160` : RIPEMD-160 해시
- `PrecompileIdentity` : 항등 함수
- `PrecompileBigModExp` : 모듈러 지수 연산
- `PrecompileBn256Add` : BN256 타원곡선 덧셈

- `PrecompileBn256ScalarMul` : BN256 스칼라 곱셈
- `PrecompileBn256Pairing` : BN256 페어링
- `PrecompileBlake2f` : Blake2f 해시

실행 상태 높이

각 `ExecutionState` 는 회로에서 차지하는 행(row) 수가 다릅니다. 이는 각 opcode가 필요로 하는 제약 조건의 복잡도에 따라 결정됩니다.

```
impl ExecutionState {
pub fn get_step_height(&self) -> usize {
EXECUTION_STATE_HEIGHT_MAP.get(self).copied()
.unwrap_or_else(|| panic!("Execution state unknown: {self:?}"))
}
}
```

높이는 `param::EXECUTION_STATE_HEIGHT_MAP` 에 정의되어 있으며, 각 gadget이 구성될 때 자동으로 계산됩니다.

Step 구조

`Step` 은 EVM 실행의 한 단계를 나타내는 구조체로, `StepState` 와 `CellManager` 로 구성됩니다.

정의 위치: `zkevm-circuits/src/evm_circuit/step.rs`

StepState

`StepState` 는 실행 단계의 상태 정보를 담고 있습니다:

```
pub(crate) struct StepState<F> {
```

```rust
// 실행 상태 선택자
pub(crate) execution_state: DynamicSelectorHalf<F>,
// 상태 변수들
pub(crate) rw_counter: Cell<F>, // Read/Write 카운터
pub(crate) call_id: Cell<F>, // 호출 ID
pub(crate) tx_id: Cell<F>, // 트랜잭션 ID
pub(crate) is_root: Cell<F>, // 루트 호출 여부
pub(crate) is_create: Cell<F>, // CREATE 호출 여부
pub(crate) block_number: Cell<F>, // 블록 번호
pub(crate) code_hash: Cell<F>, // 코드 해시
pub(crate) program_counter: Cell<F>, // 프로그램 카운터
pub(crate) stack_pointer: Cell<F>, // 스택 포인터
pub(crate) gas_left: Cell<F>, // 남은 가스
pub(crate) memory_word_size: Cell<F>, // 메모리 워드 크기
pub(crate) reversible_write_counter: Cell<F>, // 되돌릴 수 있는 쓰기 카운터
pub(crate) log_id: Cell<F>, // 로그 ID
pub(crate) end_tx: Cell<F>, // 트랜잭션 종료 여부
}
```

DynamicSelectorHalf

`DynamicSelectorHalf` 는 N개의 가능한 실행 상태 중 하나를 선택하기 위한 효율적인 선택자입니다. N/2 + 1개의 셀만 사용하여 N개의 상태를 구분할 수 있습니다.

```
pub(crate) struct DynamicSelectorHalf<F> {
count: usize, // 가능한 상태의 수
pub(crate) target_odd: Cell<F>, // 타겟이 홀수인지 (target % 2 == 1)
pub(crate) target_pairs: Vec<Cell<F>>, // 각 연속된 쌍에 속하는지
}
```

동작 원리:

- 각 상태를 짝수/홀수로 구분 (`target_odd`)
- 연속된 두 상태를 하나의 쌍으로 묶음 (`target_pairs`)
- 예: 상태 0-7을 표현하려면 4개의 셀 필요 (1개의 `target_odd` + 4개의 `target_pairs`)

CellManager

`CellManager` 는 회로의 셀 할당을 관리하는 핵심 컴포넌트입니다. 각 gadget은 `CellManager` 를 통해 필요한 셀을 요청하고, 이를 통해 회로의 높이가 자동으로 계산됩니다.

주요 기능:

- 셀 타입별 할당 관리 (StoragePhase1, StoragePhase2, Lookup 등)
- 회로 높이 추적
- 컬럼 재사용을 통한 효율적인 공간 활용

```
impl<F: Field> Step<F> {
pub(crate) fn  new(
meta: &mut ConstraintSystem<F>,
advices: [Column<Advice>; STEP_WIDTH],
offset: usize,
```

```
is_next: bool,
) -> Self {
let height = if is_next {
STEP_STATE_HEIGHT // 다음 단계는 상태만 쿼리
} else {
MAX_STEP_HEIGHT // 현재 단계는 전체 쿼리
};
let mut cell_manager = CellManager::new(meta, height, &advices, offset);
// ... StepState 구성
}
}
```

ConstraintBuilder를 통한 제약 조건 구축

`EVMConstraintBuilder` 는 각 execution gadget이 제약 조건을 정의하는 데 사용하는 핵심 도구입니다.

정의 위치: `zkevm-circuits/src/evm_circuit/util/constraint_builder.rs`

기본 구조

```
pub struct EVMConstraintBuilder<F: Field> {
pub curr: Step<F>, // 현재 단계
pub next: Step<F>, // 다음 단계
challenges: Challenges<Expression<F>>,
```

```
execution_state: ExecutionState,

constraints: Constraints<F>,

rw_counter_offset: Expression<F>,

program_counter_offset: usize,

stack_pointer_offset: i32,

// ...

}
```

주요 메서드

1. 스택 연산

```
// 스택에서 값 팝

cb.stack_pop(value.expr());


// 스택에 값 푸시

cb.stack_push(result.expr());
```

2. 메모리 연산

```
// 메모리 읽기

cb.memory_lookup(is_write.expr(), address.expr(), value.expr(), None);


// 메모리 쓰기
```

```
cb.memory_lookup(1.expr(), address.expr(), value.expr(), None);
```

3. 스토리지 연산

```
// 스토리지 읽기
cb.account_storage_read(
address.expr(),
key.expr(),
value.expr(),
tx_id.expr(),
committed_value.expr(),
);

// 스토리지 쓰기
cb.account_storage_write(
address.expr(),
key.expr(),
value.expr(),
value_prev.expr(),
tx_id.expr(),
committed_value.expr(),
reversion_info,
);
```

4. 테이블 룩업

```
// Bytecode 테이블 룩업
cb.bytecode_lookup(code_hash.expr(), program_counter.expr(), is_code.expr());


// Copy 테이블 룩업
cb.copy_table_lookup(
src_id.expr(),
src_type.expr(),
dst_id.expr(),
dst_type.expr(),
src_addr.expr(),
src_addr_end.expr(),
dst_addr.expr(),
length.expr(),
rlc_acc.expr(),
rwc_inc.expr(),
);


// Keccak 테이블 룩업
cb.keccak_table_lookup(input_rlc.expr(), input_len.expr(), output_rlc.expr());
```

5. 제약 조건 추가

```
// 기본 제약 조건
cb.require_zero("constraint name", expression);
cb.require_equal("constraint name", lhs, rhs);
cb.require_boolean("constraint name", value);

// 조건부 제약 조건
cb.condition(condition_expr, |cb| {
cb.require_equal("...", lhs, rhs);
});
```

6. 상태 전환

```
let  step_state_transition = StepStateTransition {
rw_counter: Delta(4.expr()), // rw_counter += 4
program_counter: Delta(1.expr()), // pc += 1
stack_pointer: Delta(2.expr()), // sp += 2
gas_left: Delta(-gas_cost.expr()), // gas -= gas_cost
memory_word_size: To(new_size.expr()), // memory_size = new_size
..Default::default()
};
```

StepStateTransition

상태 전환은 세 가지 방식으로 표현됩니다:

- `Same` : 값이 변하지 않음
- `Delta(expr)` : 현재 값에 `expr` 을 더함
- `To(expr)` : 값을 `expr` 로 설정
- `Any` : 제약 없음 (자유롭게 변경 가능)

ExecutionGadget 트레이트

모든 opcode 구현은 `ExecutionGadget` 트레이트를 구현합니다.

정의 위치: `zkevm-circuits/src/evm_circuit/execution.rs`

```
pub(crate) trait ExecutionGadget<F: Field> {
const NAME: &'static str;
const EXECUTION_STATE: ExecutionState;

// 제약 조건 구성
fn  configure(cb: &mut EVMConstraintBuilder<F>) -> Self;

// Witness 할당
fn  assign_exec_step(
&self,
region: &mut CachedRegion<'_, '_, F>,
offset: usize,
block: &Block,
transaction: &Transaction,
```

```
call: &Call,
step: &ExecStep,
) -> Result<(), Error>;
}
```

구현 패턴

각 gadget은 다음 패턴을 따릅니다:

1. **구조체 정의**: 필요한 셀과 서브 gadget 선언
2. **configure**: 제약 조건 구성
3. **assign_exec_step**: Witness 데이터 할당

Opcode 예시

예시 1: MULMOD

`MULMOD` opcode는 `(a * b) mod n` 을 계산합니다.

파일 위치: `zkevm-circuits/src/evm_circuit/execution/mulmod.rs`

구조체 정의

```rust
pub(crate) struct MulModGadget<F> {
same_context: SameContextGadget<F>,
words: [Word<F>; 4], // a, b, n, r
k: Word<F>, // 몫
a_reduced: Word<F>, // a mod n
```

```
d: Word<F>, // 곱셈 결과 상위 워드

e: Word<F>, // 곱셈 결과 하위 워드

modword: ModGadget<F, true>,

mul512_left: MulAddWords512Gadget<F>,

mul512_right: MulAddWords512Gadget<F>,

n_is_zero: IsZeroGadget<F>,

lt: LtWordGadget<F>,

}
```

제약 조건 구성

```
fn  configure(cb: &mut EVMConstraintBuilder<F>) -> Self {

let  opcode = cb.query_cell();

let  a = cb.query_word_rlc();

let  b = cb.query_word_rlc();

let  n = cb.query_word_rlc();

let  r = cb.query_word_rlc();

// 1. k1 * n + a_reduced == a (a를 n으로 나눈 나머지)

let  modword = ModGadget::construct(cb, [&a, &n, &a_reduced]);

// 2. a_reduced * b == d * 2^256 + e (512비트 곱셈)

let  mul512_left = MulAddWords512Gadget::construct(

cb, [&a_reduced, &b, &d, &e], None

);

// 3. k2 * n + r == d * 2^256 + e (최종 모듈로 연산)
```

```
let  mul512_right = MulAddWords512Gadget::construct(
cb, [&k, &n, &d, &e], Some(&r)
);
// 4. (r < n) 또는 n == 0 검증
let  n_is_zero = IsZeroGadget::construct(cb, sum::expr(&n.cells));
let  lt = LtWordGadget::construct(cb, &r, &n);
cb.add_constraint(
"(r < n) or (n == 0)",
1.expr() - lt.expr() - n_is_zero.expr(),
);
// 스택 연산
cb.stack_pop(a.expr());
cb.stack_pop(b.expr());
cb.stack_pop(n.expr());
cb.stack_push(r.expr());
// 상태 전환
let  step_state_transition = StepStateTransition {
rw_counter: Delta(4.expr()),
program_counter: Delta(1.expr()),
stack_pointer: Delta(2.expr()),
gas_left: Delta(-OpcodeId::MULMOD.constant_gas_cost().expr()),
..Default::default()
};
```

```
let  same_context = SameContextGadget::construct(cb, opcode, step_state_transiti

Self { /* ... */ }

}
```

검증 로직:

1. `a` 를 `n` 으로 나눈 나머지 `a_reduced` 계산
2. `a_reduced * b` 를 512비트로 계산하여 `d * 2^256 + e` 로 표현
3. 이를 다시 `n` 으로 나눈 나머지가 `r` 임을 검증
4. `r < n` 또는 `n == 0` 조건 확인

예시 2: RETURNDATACOPY

`RETURNDATACOPY` opcode는 이전 호출의 반환 데이터를 메모리로 복사합니다.

파일 위치: `zkevm-circuits/src/evm_circuit/execution/returndatacopy.rs`

구조체 정의

```
pub(crate) struct ReturnDataCopyGadget<F> {

same_context: SameContextGadget<F>,

last_callee_id: Cell<F>, // 마지막 호출된 컨트랙트 ID

return_data_offset: Cell<F>, // 반환 데이터 오프셋

return_data_size: Cell<F>, // 반환 데이터 크기

dst_memory_addr: MemoryAddressGadget<F>, // 목적지 메모리 주소

check_overflow_gadget: CommonReturnDataCopyGadget<F>,

memory_expansion: MemoryExpansionGadget<F, 1, N_BYTES_MEMORY_WORD_SIZE>,
```

```
memory_copier_gas: MemoryCopierGasGadget<F, { GasCost::COPY }>,
copy_rwc_inc: Cell<F>, // Copy 테이블 RW 카운터 증가량
}
```

제약 조건 구성

```
fn configure(cb: &mut EVMConstraintBuilder<F>) -> Self {
let opcode = cb.query_cell();
let dest_offset = cb.query_cell_phase2();
let return_data_size = cb.query_cell();
let size = cb.query_word_rlc();
// 1. 오버플로우 검사
let check_overflow_gadget = CommonReturnDataCopyGadget::construct(
cb, return_data_size.expr(), false.expr()
);
// 2. 스택에서 파라미터 팝
cb.stack_pop(dest_offset.expr());
cb.stack_pop(check_overflow_gadget.data_offset().expr());
cb.stack_pop(size.expr());
// 3. Call Context에서 반환 데이터 정보 조회
let last_callee_id = cb.query_cell();
let return_data_offset = cb.query_cell();
cb.call_context_lookup(
false.expr(), None,
```

```
CallContextFieldTag::LastCalleeId,
last_callee_id.expr(),
);
cb.call_context_lookup(
false.expr(), None,
CallContextFieldTag::LastCalleeReturnDataOffset,
return_data_offset.expr(),
);
cb.call_context_lookup(
false.expr(), None,
CallContextFieldTag::LastCalleeReturnDataLength,
return_data_size.expr(),
);
// 4. 메모리 주소 계산 및 확장
let  dst_memory_addr = MemoryAddressGadget::construct(cb, dest_offset, size);
let  memory_expansion = MemoryExpansionGadget::construct(
cb, [dst_memory_addr.end_offset()]
);
let  memory_copier_gas = MemoryCopierGasGadget::construct(
cb, dst_memory_addr.length(), memory_expansion.gas_cost()
);
// 5. Copy 테이블 룩업 (복사할 데이터가 있을 때만)
let  copy_rwc_inc = cb.query_cell();
```

```
cb.condition(dst_memory_addr.has_length(), |cb| {
cb.copy_table_lookup(
last_callee_id.expr(),
CopyDataType::Memory.expr(),
cb.curr.state.call_id.expr(),
CopyDataType::Memory.expr(),
return_data_offset.expr() + data_offset.expr(),
return_data_offset.expr() + return_data_size.expr(),
dst_memory_addr.offset(),
dst_memory_addr.length(),
0.expr(),
copy_rwc_inc.expr(),
);
});
// 6. 상태 전환
let  step_state_transition = StepStateTransition {
rw_counter: Delta(cb.rw_counter_offset()),
program_counter: Delta(1.expr()),
stack_pointer: Delta(3.expr()),
gas_left: Delta(
-(OpcodeId::RETURNDATACOPY.constant_gas_cost().expr()
+ memory_copier_gas.gas_cost())
),
```

```
memory_word_size: To(memory_expansion.next_memory_word_size()),
..Default::default()
};
let  same_context = SameContextGadget::construct(cb, opcode, step_state_transiti
Self { /* ... */ }
}
```

검증 로직:

1. 스택에서 `dest_offset`, `data_offset`, `size` 팝
2. Call Context에서 마지막 호출의 반환 데이터 정보 조회
3. 반환 데이터 범위 검증 (오버플로우 체크)
4. 메모리 확장 및 가스 비용 계산
5. Copy 테이블을 통해 실제 복사 연산 검증
6. 상태 전환 (PC, SP, 가스, 메모리 크기)

ExecutionConfig 구조

`ExecutionConfig` 는 모든 execution gadget을 포함하는 최상위 구성입니다.

정의 위치: `zkevm-circuits/src/evm_circuit/execution.rs`

주요 컴포넌트

```
pub(crate) struct ExecutionConfig<F> {
// 선택자
```

```
q_usable: Column<Fixed>, // 사용 가능한 행

q_step: Column<Advice>, // 단계 시작 행

q_step_first: Selector, // 첫 번째 단계

q_step_last: Selector, // 마지막 단계

// 단계 관리

step: Step<F>,

num_rows_until_next_step: Column<Advice>,

num_rows_inv: Column<Advice>,

// Advice 컬럼

advices: [Column<Advice>; STEP_WIDTH],

// 높이 맵

height_map: HashMap<ExecutionState, usize>,

stored_expressions_map: HashMap<ExecutionState, Vec<StoredExpression<F>>>,

// 내부 상태 gadget

begin_tx_gadget: Box<BeginTxGadget<F>>,

end_tx_gadget: Box<EndTxGadget<F>>,

end_block_gadget: Box<EndBlockGadget<F>>,

padding_gadget: Box<PaddingGadget<F>>,

// Opcode gadget (일부만 표시)

add_sub_gadget: Box<AddSubGadget<F>>,

mulmod_gadget: Box<MulModGadget<F>>,

returndatacopy_gadget: Box<ReturnDataCopyGadget<F>>,

// ... 모든 opcode gadget
```

```
// 에러 gadget
error_stack: Box<ErrorStackGadget<F>>,
error_oog_constant: Box<ErrorOOGConstantGadget<F>>,
// ... 모든 에러 gadget
// 프리컴파일 gadget
precompile_ecrecover_gadget: Box<EcrecoverGadget<F>>,
precompile_sha2_gadget: Box<SHA256Gadget<F>>,
// ... 모든 프리컴파일 gadget
}
```

Gadget 구성 프로세스

각 gadget은 다음 과정을 거쳐 구성됩니다:

```
fn configure_gadget<G: ExecutionGadget<F>>(
meta: &mut ConstraintSystem<F>,
advices: [Column<Advice>; STEP_WIDTH],
q_usable: Column<Fixed>,
q_step: Column<Advice>,
// ...
) -> G {
// 1. 최대 높이로 먼저 구성하여 실제 높이 파악
let height = {
let dummy_step_next = Step::new(meta, advices, MAX_STEP_HEIGHT, true);
```

```
let mut cb = EVMConstraintBuilder::new(
step_curr.clone(),
dummy_step_next,
challenges,
G::EXECUTION_STATE,
);
cb.annotation(G::NAME, |cb| G::configure(cb));
let (_, _, _, height) = cb.build();
height
};
// 2. 실제 높이로 다시 구성
let step_next = Step::new(meta, advices, height, true);
let mut cb = EVMConstraintBuilder::new(
step_curr.clone(),
step_next.clone(),
challenges,
G::EXECUTION_STATE,
);
let gadget = cb.annotation(G::NAME, |cb| G::configure(cb));
// 3. 제약 조건 등록
let (state_selector, constraints, stored_expressions, _) = cb.build();
// 4. 높이 맵에 저장
height_map.insert(G::EXECUTION_STATE, height);
```

```
gadget

}
```

상태 전환 검증

EVM Circuit은 실행 상태 간의 전환이 올바른지 검증합니다:

```
meta.create_gate("Constrain state machine transitions", |meta| {

// EndTx는 BeginTx 또는 EndInnerBlock으로만 전환 가능

// EndInnerBlock은 BeginTx, EndInnerBlock, Padding으로 전환 가능

// Padding은 Padding 또는 EndBlock으로만 전환 가능

// BeginTx는 EndTx 또는 EndInnerBlock에서만 올 수 있음

// EndBlock은 Padding에서만 올 수 있음

// ...

});
```

테이블 룩업

EVM Circuit은 다양한 테이블과 룩업을 통해 다른 회로와 데이터를 공유합니다.

룩업 테이블 종류

1. **Fixed Table**: 고정된 값들 (비트 연산 테이블 등)
2. **Tx Table**: 트랜잭션 데이터
3. **RW Table**: Read/Write 연산 (State Circuit과 공유)
4. **Bytecode Table**: 바이트코드 데이터

5. **Block Table**: 블록 메타데이터
6. **Copy Table**: 복사 연산 검증
7. **Keccak Table**: Keccak 해시 결과
8. **SHA256 Table**: SHA256 해시 결과
9. **Exp Table**: 지수 연산 결과
10. **Sig Table**: 서명 검증 결과
11. **ModExp Table**: 모듈러 지수 연산 결과
12. **Ecc Table**: 타원곡선 연산 결과
13. **PowOfRand Table**: 랜덤 값의 거듭제곱

룩업 구성

```
fn configure_lookup(
meta: &mut ConstraintSystem<F>,
// ... 모든 테이블
challenges: &Challenges<Expression<F>>,
cell_manager: &CellManager<F>,
) {
for column in cell_manager.columns().iter() {
if let CellType::Lookup(table) = column.cell_type {
meta.lookup_any(table_name, |meta| {
let table_expressions = match table {
Table::Fixed => fixed_table,
Table::Tx => tx_table,
```

```
Table::Rw => rw_table,
// ...
}.table_exprs(meta);
vec![(
column.expr(),
rlc::expr(&table_expressions, challenges.lookup_input()),
)]
});
}
}
}
```

룩업은 RLC (Random Linear Combination)를 사용하여 여러 컬럼을 하나의 값으로 압축합니다.

Witness 할당

Witness 할당은 실제 실행 데이터를 회로에 할당하는 과정입니다.

블록 할당 프로세스

```
pub fn assign_block(
&self,
layouter: &mut impl Layouter<F>,
block: &Block,
challenges: &Challenges<Value<F>>,
```

```
) -> Result<EvmCircuitExports<Assigned<F>>, Error> {
// 1. 영역 높이 계산
let region1_height = self.get_num_rows_required_no_padding(block);
let region3_height = ExecutionState::EndBlock.get_step_height() + 1;
let region2_height = if no_padding {
1
} else {
evm_rows - region3_height - region1_height
};
// 2. 실제 단계 할당 (region1)
for (tx_idx, tx) in block.txs.iter().enumerate() {
for (step_idx, step) in tx.steps.iter().enumerate() {
let height = step.execution_state.get_step_height();
self.assign_exec_step(
&mut region,
offset,
block,
transaction,
call,
step,
height,
Some(next),
challenges,
```

```
)?;
self.assign_q_step(&mut  region, &inverter, offset, height)?;
offset += height;
}
}
// 3. 패딩 할당 (region2)
self.assign_same_exec_step_in_range(
&mut  region,
0,
region_height,
block,
&dummy_tx,
&last_call,
padding_step,
1,
challenges,
)?;
// 4. EndBlock 할당 (region3)
self.assign_exec_step(
&mut  region,
offset,
block,
&dummy_tx,
```

```
&last_call,
end_block_step,
height,
None,
challenges,
)?;
Ok(EvmCircuitExports { withdraw_root })
}
```

개별 단계 할당

```
fn  assign_exec_step(
&self,
region: &mut Region<'_, F>,
offset: usize,
block: &Block,
transaction: &Transaction,
call: &Call,
step: &ExecStep,
height: usize,
next: Option<(&Transaction, &Call, &ExecStep)>,
challenges: &Challenges<Value<F>>,
) -> Result<(), Error> {
```

```
// 1. 현재 단계 상태 할당
self.step.assign_exec_step(region, offset, block, transaction, call, step)?;
// 2. 다음 단계 상태 할당 (있는 경우)
if let Some((next_tx, next_call, next_step)) = next {
// ...
}
// 3. 실행 상태에 따른 gadget 호출
macro_rules! assign_exec_step {
($gadget:expr) => {
$gadget.assign_exec_step(region, offset, block, transaction, call, step)?
};
}
match step.execution_state {
ExecutionState::MULMOD => assign_exec_step!(self.mulmod_gadget),
ExecutionState::RETURNDATACOPY => assign_exec_step!(self.returndatacopy_gadget),
ExecutionState::ADD_SUB => assign_exec_step!(self.add_sub_gadget),
// ... 모든 실행 상태
_ => unreachable!(),
}
Ok(())
}
```

회로 크기 및 최적화

행 수 계산

```
pub fn get_num_rows_required(block: &Block) -> usize {
let mut num_rows = 0;
for transaction in &block.txs {
for step in &transaction.steps {
num_rows += step.execution_state.get_step_height();
}
}
// EndBlock과 최소 1개의 미사용 행 추가
num_rows += 1;
num_rows += ExecutionState::EndBlock.get_step_height();
num_rows += 1;
num_rows
}
```

컬럼 구성

EVM Circuit의 advice 컬럼은 다음과 같이 구성됩니다:

- **Phase 3 (Lookup)**: EVM_LOOKUP_COLS개 - 테이블 룩업용
- **Phase 2**: N_PHASE2_COLUMNS개 - 2단계 연산용
- **Phase 1**: 나머지 - 1단계 연산용

총 STEP_WIDTH 개의 컬럼을 사용합니다.

주요 특징

1. 동적 높이

각 실행 단계는 필요한 만큼의 행만 사용합니다. 이는 num_rows_until_next_step 컬럼을 통해 관리됩니다:

```
meta.create_gate("q_step", |meta| {

let  q_step = meta.query_advice(q_step, Rotation::cur());

let  num_rows_left_cur = meta.query_advice(num_rows_until_next_step, Rotation::c

let  num_rows_left_next = meta.query_advice(num_rows_until_next_step, Rotation::

// 단계가 활성화되지 않은 경우, 카운터는 1씩 감소

cb.condition(not::expr(q_step.clone()), |cb| {

cb.require_equal(

"num_rows_left_cur := num_rows_left_next + 1",

num_rows_left_cur.clone(),

num_rows_left_next + 1.expr(),

);

});

// q_step은 num_rows_until_next_step == 0일 때만 활성화

let  is_zero = 1.expr() - (num_rows_left_cur.clone() * num_rows_left_inverse);

cb.require_equal("q_step == is_zero", q_step, is_zero);
```

```
});
```

2. 병렬 할당

Witness 할당은 병렬로 수행되어 성능을 향상시킵니다:

```
let (chunk_size, chunk_num) = chunking_fn("region1", step_assignments.len(), 50)
layouter.assign_regions(
|| "Execution step region1",
region1_is_first_time
.iter_mut()
.map(|(chunk_idx, is_first_time)| {
|mut  region: Region<'_, F>| {
// 청크별 병렬 할당
// ...
}
})
.collect_vec(),
)?;
```

3. 상태 전환 검증

모든 실행 상태 간 전환은 명시적으로 검증됩니다:

- `EndTx` → `BeginTx` 또는 `EndInnerBlock`

- `EndInnerBlock` → `BeginTx`, `EndInnerBlock`, 또는 `Padding`
- `Padding` → `Padding` 또는 `EndBlock`
- 첫 번째 단계는 `BeginTx` 또는 `Padding` 이어야 함
- 마지막 단계는 `EndBlock` 이어야 함

4. 에러 처리

EVM Circuit은 다양한 에러 상태를 지원합니다:

- **스택 에러**: 언더플로우/오버플로우
- **가스 부족**: 각 연산 타입별 세분화된 에러
- **쓰기 보호**: STATICCALL 내에서의 상태 변경 시도
- **유효하지 않은 점프**: JUMPDEST가 아닌 위치로의 점프
- **반환 데이터 범위 초과**: RETURNDATACOPY에서 범위 초과
- **코드 저장 에러**: 컨트랙트 생성 시 코드 크기 초과

각 에러는 별도의 `ExecutionState` 와 gadget으로 구현됩니다.

관련 문서

이전/다음

- **이전**: 데이터 흐름 - 전체 데이터 흐름
- **다음**: State Circuit - 상태 관리 회로

같은 섹션 (핵심 회로)

- State Circuit - RW 테이블을 통한 상태 관리
- Bytecode Circuit - 바이트코드 검증

- Keccak Circuit - 해시 함수 검증
- Tx Circuit - 트랜잭션 검증
- Sig Circuit - 서명 검증
- Copy Circuit - 복사 연산 검증

관련 시스템

- Bus-Mapping 개요 - 실행 트레이스 변환
- Opcode 처리 - Opcode별 ExecStep 생성
- 테이블 시스템 - 회로 간 데이터 공유
- RW Table - Read-Write 연산 테이블

구현 세부사항

- Halo2 통합 - Halo2 API 사용
- 제약 조건 구축 - ConstraintBuilder 패턴
- Gadget 시스템 - 재사용 가능한 컴포넌트
- CellManager - 셀 할당 최적화

참고 자료

- **코드 위치**:
- `zkevm-circuits/src/evm_circuit.rs` : 메인 회로 정의
- `zkevm-circuits/src/evm_circuit/execution.rs` : 실행 구성
- `zkevm-circuits/src/evm_circuit/step.rs` : 단계 정의
- `zkevm-circuits/src/evm_circuit/execution/` : 개별 opcode 구현

- `zkevm-circuits/src/evm_circuit/util/constraint_builder.rs` : 제약 조건 빌더
- **주요 개념**:
- ExecutionState: 각 opcode 또는 내부 상태
- Step: 실행 단계의 상태 정보
- CellManager: 셀 할당 관리
- ConstraintBuilder: 제약 조건 구축
- ExecutionGadget: Opcode 구현 트레이트